Практическая работа: T-SQL — управляющие конструкции, индексы и нормализация

Что сдать в отчёте

1. **Схема базы данных** — SQL-скрипт создания всех таблиц (CREATE TABLE), ключей, ограничений и связей. Схема должна быть в 3NF.
2. **Скрипт заполнения данными** — все операторы INSERT для тестовых данных.
3. **Скрипты решений заданий 1–10** — каждый запрос или батч в отдельном блоке с указанием номера задания. Скрипты должны выполняться без ошибок.
4. **Объяснения по текстовым заданиям:**
   1) Задание 1 — краткое описание каждого индекса (1–2 предложения на индекс).
   2) Задание 9 — что произошло во втором батче и почему (2–3 предложения).
   3) Задание 10 — запрос + объяснение связи нормализации и возможности решения (2–3 предложения).
5. **Скриншоты планов выполнения:**
   1) Задание 8 — планы для обоих вариантов (EXISTS и IN).
   2) Задание 11 — планы до и после создания покрывающего индекса.
6. **Вывод** — 3–5 предложений: что освоено, с какими конструкциями T-SQL возникли сложности, где применимы изученные приёмы в реальной разработке.

Предметная область: книжный интернет-магазин.

Часть 1. Проектирование схемы (нормализация)

Задание. Спроектируй схему БД для книжного магазина в 3NF. Обязательно должны быть:

- не менее 3 таблиц;
- связь «многие-ко-многим» (книга — категории);
- отдельная таблица для позиций заказа (чтобы один заказ мог содержать несколько книг).

Создай таблицы с корректными типами данных, первичными и внешними ключами, ограничениями (NOT NULL, CHECK, UNIQUE и т. п.). Не используй хранение списков через запятую и не дублируй данные.

Часть 2. Заполнение данными

Задание. Наполни таблицы тестовыми данными (INSERT):

- минимум 6 книг, 4 автора, 4 категории;
- свяжи книги с категориями (учти, что у одной книги может быть несколько категорий);
- добавь минимум 5 клиентов и 8 заказов;
- заполни позиции заказов так, чтобы были заказы с несколькими книгами и разные количества товаров.

Убедись, что данные согласованы: например, в позициях заказов не должно быть книг, которых нет в таблице Books.

Часть 3. Практические задания

Задание 1. Индексы

Создай:

- кластерный индекс на таблице Orders по полю OrderDate;
- некластерный покрывающий индекс на таблице Books по AuthorId, включив в него поля для вывода (Title, Price, StockQty);
- фильтрованный некластерный индекс на Orders только для заказов со статусом 'Pending'.

После создания индексов опиши, как каждый из них помогает ускорить типовые запросы к этим таблицам (1–2 предложения на индекс).

Задание 2. TOP с WITH TIES

Напиши запрос, который выводит топ-3 самых дорогих книг. Если на границе третьего места есть книги с одинаковой ценой, они все должны попасть в результат. Не используй подзапросы и оконные функции — только TOP и WITH TIES.

Задание 3. IF…ELSE + EXISTS

Проверь, есть ли книги с нулевым остатком на складе. Если такие есть — выведи сообщение «Внимание: есть книги с нулевым остатком!» и список их названий. Если таких нет — выведи «Все книги есть в наличии». Используй конструкцию IF EXISTS, а не COUNT(*).

Задание 4. TRY…CATCH + THROW

Подготовь батч, который пытается вставить заказ с несуществующим CustomerId (например, 999). Перехвати ошибку в блоке CATCH, выведи номер ошибки и текст сообщения. В конце повторно выброси ошибку через THROW, чтобы показать, что ошибка не «проглочена».

Задание 5. OUTPUT при UPDATE

Обнови статус всех заказов со значением 'Pending' на 'Shipped'. С помощью конструкции OUTPUT выведи: OrderId, старый статус и новый статус.

Задание 6. Переменные и WHILE

Объяви переменную @threshold = 10. С помощью цикла WHILE уменьшай её на 1 на каждой итерации и выводи количество книг, у которых StockQty < @threshold. Цикл должен остановиться, когда @threshold станет равен 0. На каждой итерации выводи строку вида: «Порог: X, книг меньше порога: Y».

Задание 7. MERGE

Создай временную таблицу-источник BookStockStaging с полями: BookId, NewStockQty, Title, AuthorId, Price. Заполни её двумя строками: одна — для существующей книги (обновление), другая — для новой книги (вставка). С помощью оператора MERGE синхронизируй запасы в таблице Books: обнови значения для существующих книг и добавь новые, если их нет. Выведи результат через OUTPUT $action, inserted.BookId, deleted.BookId.

Задание 8. EXISTS vs IN

Найди всех авторов, у которых хотя бы одна книга была заказана. Реши задачу двумя способами:

1. через EXISTS;
2. через IN.

Для каждого варианта напиши только сам запрос (без пояснений и планов выполнения).

Задание 9. GO и область видимости

Напиши два отдельных батча, разделённых GO:

- в первом — объяви переменную @city = 'Москва' и выведи всех клиентов из этого города;
- во втором — попробуй использовать ту же переменную @city в запросе к таблице клиентов.

Запусти оба батча и опиши результат: что произойдёт во втором батче и почему. Ответ дай в виде 2–3 предложений.

Задание 10. Клиенты, купившие книги из 3+ категорий в одном заказе

Найди всех клиентов, которые в одном заказе купили книги, относящиеся к трём или более разным категориям. Выведи имя клиента, OrderId и количество уникальных категорий.

Важно: это задание невозможно выполнить корректно, если схема не нормализована:

- если нет таблицы позиций заказа (OrderItems) — невозможно хранить несколько книг в одном заказе;
- если категории хранятся в виде строки через запятую в поле книги — невозможно посчитать уникальные категории через JOIN.

Если схема спроектирована правильно, запрос выполнится. Если нет — он либо не сработает, либо даст неверный результат. Напиши запрос и в 2–3 предложениях объясни, как именно нормализация схемы позволяет решить эту задачу.

Задание 11. Index Seek / Key Lookup

Посмотри на запрос: SELECT AuthorId, Title, Price FROM Books WHERE AuthorId = 1.
Опиши, какой тип операции (Index Seek / Key Lookup) ты ожидаешь увидеть в плане выполнения:

- до создания покрывающего индекса из Задания 1;
- после создания этого индекса.

Ответ дай в виде двух коротких пунктов (по 1 фразе).
